임베디드 시스템

Embedded Systems

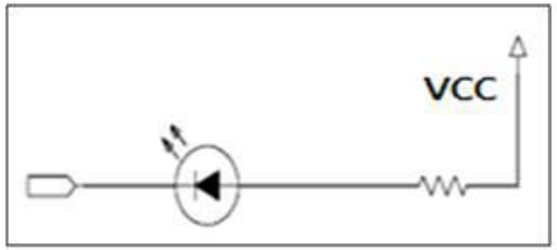
RGB LED | Active Buzzer | Push Switch

강원대학교 컴퓨터공학과 권용훈

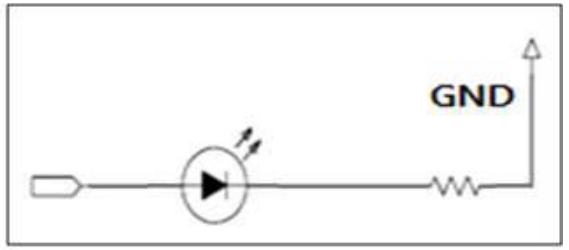
RGB LED(1/4)

LED(Light Emitting Diode)

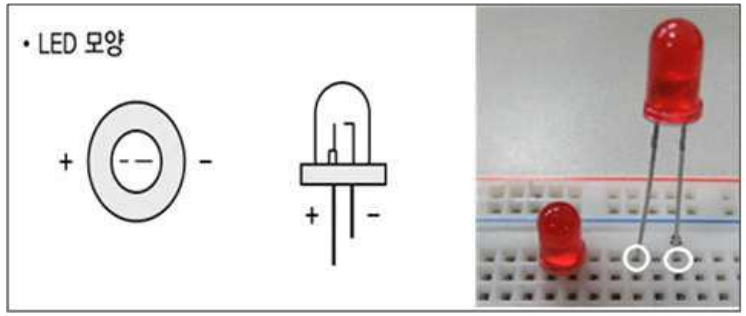
- 타입에 따라 에노드(Anode)와 캐소드(Cathode)로 구분



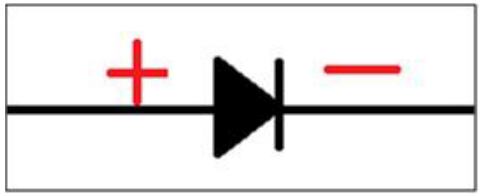
[Anode Type]



[Cathode Type]



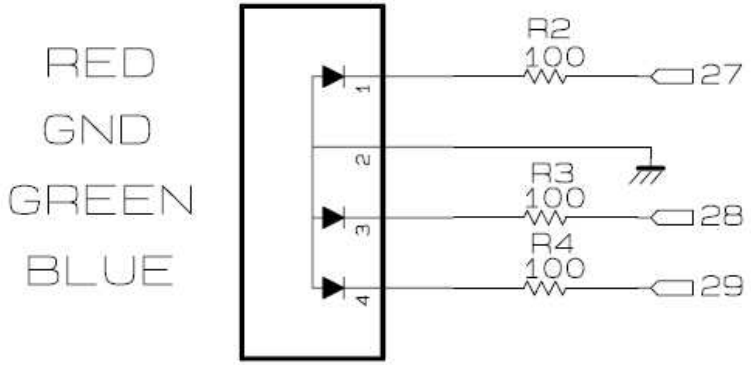
연결 방식	애노드		캐소드	
동작 신호	LED ON	=> LOW	LED ON	=> HIGH
	LED OFF	=> HIGH	LED OFF	=> LOW



RGB LED(2/4)

LED 연결 핀

키트의 RGB-LED 모듈



BCM	wPi	Name	Mode	V	Physical	V	Mode	Name	wPi	BCM
6	22	GPIO.22	IN	1	31 || 32	0	IN	GPIO.26	26	12
13	23	GPIO.23	IN	0	33 || 34			0v		
19	24	GPIO.24	IN	0	35 || 36	0	IN	GPIO.27	27	16
26	25	GPIO.25	IN	0	37 || 38	0	IN	GPIO.28	28	20
		0v			39 || 40	0	IN	GPIO.29	29	21

Pi 3B

GPIO 핀 맵에서 wiringPi 핀 – 27,28,29

RGB LED(3/4)

LED 제어 코드

[EX_RGB_LED.c]

```
01 //- 센서 제어 라이브러리 ----------------------------------------------------------
02 #include <stdio.h>
03 #include <wiringPi.h>
04
05 //- 센서 연결 핀 선언 ----------------------------------------------------------
06 #define R_LED_PIN                    27    //WPI
07 #define G_LED_PIN                    28    //WPI
08 #define B_LED_PIN                    29    //WPI
09
10 //- 프로그램 실행 함수 ----------------------------------------------------------
11 int main(void){
12
13         //- wiringPi 초기화
14         if( wiringPiSetup() == -1)
15         {
16                 printf("Setup Fail ~!!");
17                 return -1;
18         }
19
20         //- 센서 초기화
21         pinMode(R_LED_PIN,   OUTPUT);
22         pinMode(G_LED_PIN,   OUTPUT);
23         pinMode(B_LED_PIN,   OUTPUT);
24
25         //- 삼색 LED 제어
26         while(1)
27         {
28                 digitalWrite(R_LED_PIN, HIGH) ;
29                 digitalWrite(B_LED_PIN, LOW) ;
30                 printf("RED LED ON ~₩n");
31                 delay(1000);
32                 digitalWrite(R_LED_PIN, LOW) ;
33                 digitalWrite(G_LED_PIN, HIGH) ;
34                 printf("GREEN LED OFF ~₩n");
35                 delay(1000);
36
37                 digitalWrite(G_LED_PIN, LOW) ;
38                 digitalWrite(B_LED_PIN, HIGH) ;
39                 printf("BLUE LED ON ~₩n");
40                 delay(1000);
4 1
42                 digitalWrite(R_LED_PIN, LOW) ;
43                 digitalWrite(G_LED_PIN, LOW) ;
44                 digitalWrite(R_LED_PIN, LOW) ;
45                 printf("RGB LED OFF ~₩n");
46                 delay(2000);
47     }
48     return 0;
49 }
```

RGB LED(4/4)

LED 제어 코드

- 코드 빌드
  - 형식: gcc –l[라이브러리 이름] [파일명] –o [실행 파일명]

```
pi@raspberrypi:~/Example/GPIO $ gcc -lwiringPi EX_RGB_LED.c -o RGB_LED
pi@raspberrypi:~/Example/GPIO $ ls
EX_RGB_LED.c  RGB_LED
pi@raspberrypi:~/Example/GPIO $
```

- 실행하기
  - 형식 : ./[실행 파일명]

```
pi@raspberrypi:~/Example/GPIO $ ./RGB_LED
Blue LED ON ~
Green LED OFF ~
Red LED ON ~
```

- 실행 종료하기

```
Red LED OFF ~
Blue LED ON ~
Green LED OFF ~
Red LED ON ~
Red LED OFF ~
^C
pi@raspberrypi:~/Example/GPIO $
```

실행 종료: Ctrl + C

Active Buzzer(1/3)

Buzzer

- 금속판을 떨리게 해서 소리를 출력하는 부품으로 단순음만 출력

	능동 부저 (Active Buzzer)	수동 부저 (Passive Buzzer)
특징	-내장된 회로 존재 -전원만 인가해주면 소리 출력	-내장 회로 없음 -출력 주파수 조절로 소리 출력
용도	-음성 출력용 불가능 -전자기기 내부 음향 신호출력용	-전자시계 알람 멜로디 -전화기의 벨소리
모양	-스티커로 양극/음극 표시 -하부에 고무로 마무리	-하부를 PCB형태로 기판이 보임

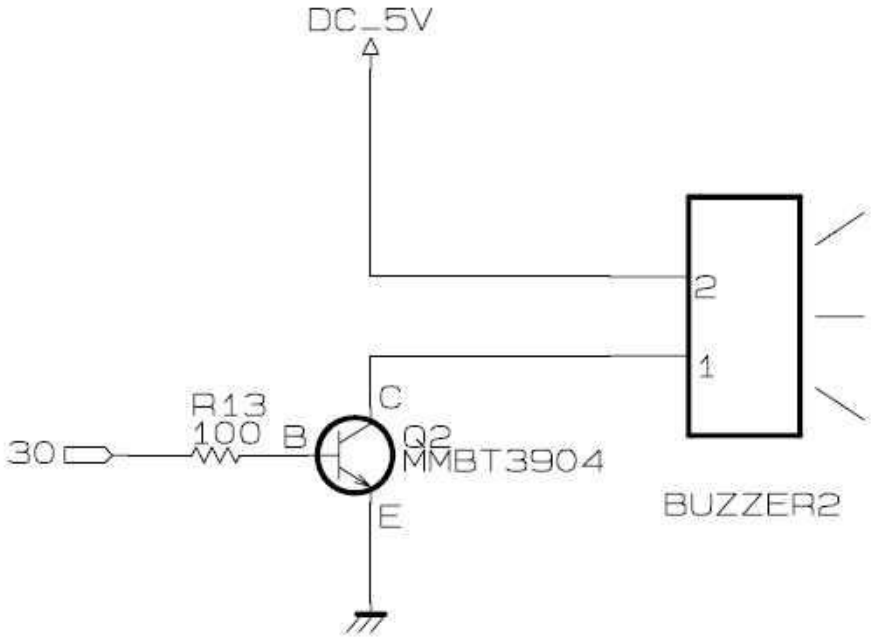
Active Buzzer(2/3)

Active Buzzer 연결 핀

		0v			25	26	1	IN	CE1	11	7
0	30	SDA.0	IN	1	27	28	1	IN	SCL.0	31	1
5	21	GPIO.21	IN	1	29	30			0v		
6	22	GPIO.22	IN	1	31	32	0	IN	GPIO.26	26	12
13	23	GPIO.23	IN	0	33	34			0v		
19	24	GPIO.24	IN	0	35	36	0	IN	GPIO.27	27	16
26	25	GPIO.25	IN	0	37	38	0	IN	GPIO.28	28	20
		0v			39	40	0	IN	GPIO.29	29	21
BCM	wPi	Name	Mode	V	Physical		V	Mode	Name	wPi	BCM

Pi 3B

GPIO 핀 맵에서 wiringPi 핀 – 30



Active Buzzer 회로 구조

Active Buzzer(3/3)

[EX_A_Buzzer.c]

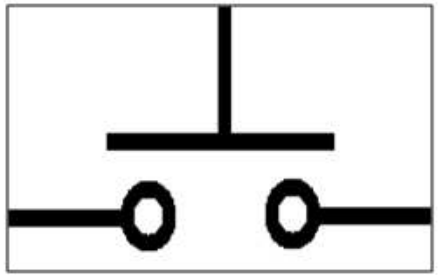
```
//- 관련 라이브러리 추가-----------------------------------------------------------------
#include <wiringPi.h>
#include <stdlib.h>

//- 핀 번호 선언 ---------------------------------------------------------------------
#define BUZ_A_PIN                    30          //- wiringPi

//- Buzzer 제어 기능 구현 -------------------------------------------------------------
int main(void)
{
    //- wiringPi 초기화
    if( wiringPiSetup() == -1)
    {cl
                printf("Setup Fail ~!!");
                return -1;
    }
    //- 센서 초기화
    pinMode(BUZ_A_PIN, OUTPUT);

    while(1)
    {
                digitalWrite(BUZ_A_PIN, HIGH);
                delay(1000);

                digitalWrite(BUZ_A_PIN, LOW);
                delay(1000);
    }
}
```

Push Switch(1/3)

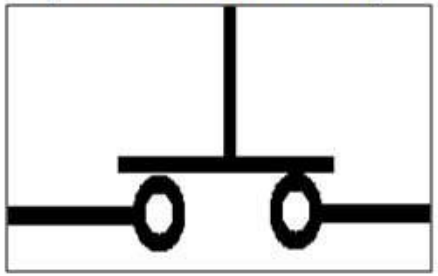
Push Switch

- 버튼을 눌러 회로를 절단 또는 접속시키는 스위치

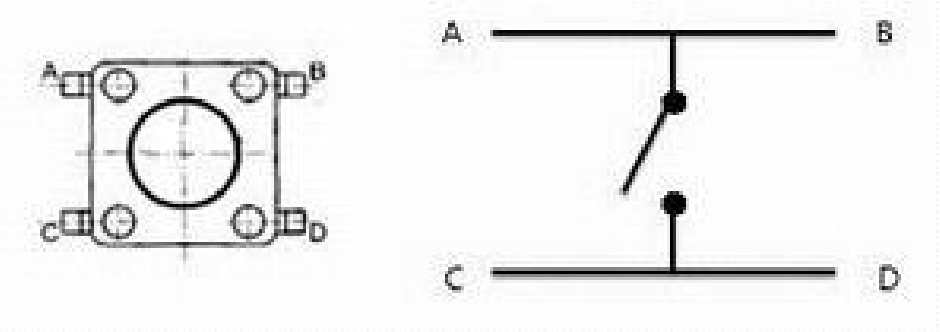
[회로 절단 Push UP]



[회로 접속 Push DOWN]



- 회로가 연결되면 저류가 흘러서 지나갈 수 있고,
  회로가 끊겨 있다면 전류가 흘러가지 못한다.



스위치의 회로 구조

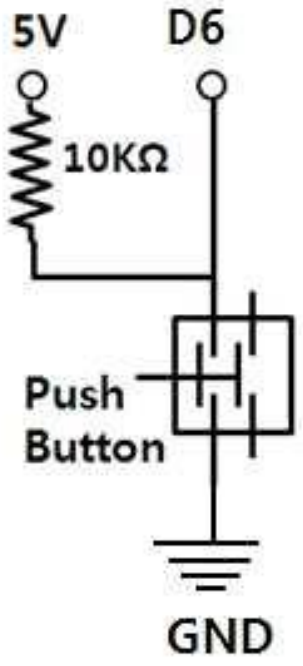
Push Switch(2/3)

방식에 따른 Push Switch 연결 구조

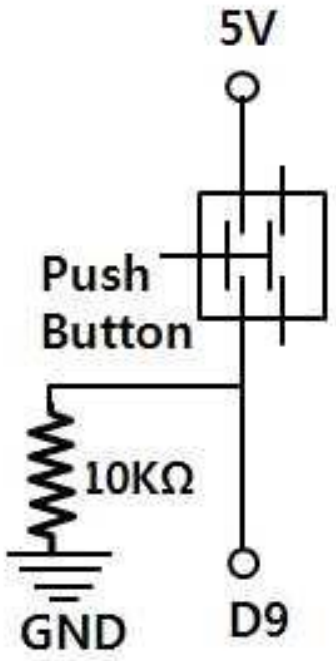
- 플로팅(floating): 전류가 흐르지 않는 상태임에도 해당 스위치에 전류 흐름 여부를 알 수 없는 상태
- 저항은 해당 현상을 방지하기 위한 장치
- 저항의 위치에 따라 풀 업과 풀 다운으로 구분한다.

풀업(Pull Up) 방식		
연결	저 항	5V와 입력 핀
	푸시버튼	접지 GND
동작	입력 O	LOW(OFF)
	입력 X	**HIGH(ON)**

풀다운(Pull Down) 방식		
연결	저 항	GND와 입력 핀
	푸시버튼	전원 5V
동작	입력 O	HIGH(ON)
	입력 X	**LOW(OFF)**



Pull up



Pull down

Push Switch(3/3)

```
#include <stdio.h>
#include <stdlib.h>
#include <wiringPi.h>
#define PUSHSWL 0
#define PUSHSWR 7

int main(){

    //wiringPi 초기화
    if(wiringPiSetup()==-1){
        printf("Set up fail~!");
        return -1;
    }

    pinMode(PUSHSWL,INPUT);
    pinMode(PUSHSWR,INPUT);

    while(1){
        if(digitalRead(PUSHSWL)==LOW){
            printf("Left Switch Pushed!\n");
            delay(300);
        }else if(digitalRead(PUSHSWR)==LOW){
            printf("Right Switch Pushed!\n");
            delay(300);
        }
    }
    return 0;
}
```

실습

기본 예제 수행

- EX_RGB_LED.c
- EX_A_BUZZER.c

실습 과제 수행

- 과제 1
- 과제 2

고생하셨습니다

Thank you!